Project 13 — N8N Workflow

Complexity Requirements & Submission Criteria

You have freedom to choose your use case, but your workflow must meet the following conditions to demonstrate mastery of the concepts covered in this course.

Minimum Complexity Requirements

Your workflow must include **at least three of the following**:

1. **Conditional Logic**

    - At least two branching decision points in your workflow
    - Different paths based on data evaluation (not just error handling)
    - Example: Route tasks differently based on priority, complexity, or data type

2. **Data Transformation**

    - Use N8N's data transformation capabilities (function nodes, mappings, filters)
    - Not just pass-through — actively reshape, aggregate, or filter data between steps
    - Example: Transform API responses into a structured format for downstream consumption

3. **Multiple External Integrations**

    - Connect to at least two external APIs or services (Slack, Gmail, Google Sheets, GitHub, etc.)
    - Plus integration with your own backend (the chatbot/database you built in earlier projects)
    - Example: Fetch data from an API, process it with your backend, post results to Slack

4. **Database or File I/O**

    - Read from or write to your Supabase database, Google Sheets, or filesystem
    - Demonstrate data persistence or history tracking
    - Example: Log workflow results to a database table for audit trails

5. **Error Handling & Retry Logic**

    - Implement try-catch nodes or error branches
    - Define fallback behavior if a step fails
    - Example: If an API call fails, send an alert and retry, or use cached data

6. **Scheduled or Triggered Execution**

    - Workflow is triggered by a schedule (cron), webhook, or manual trigger
    - Not just linear top-to-bottom execution
    - Example: Run daily at 9 AM or trigger when a message arrives in a queue

---

Integration with Your Course Work

Your workflow **must** interact with at least one of the following:

- **Your chatbot backend** — e.g., trigger workflow actions based on chat messages or user queries
- **Your database (Supabase)** — e.g., read user data, store workflow results
- **Your AI models** — e.g., use LiteLLM within the workflow
- **Your LLM agents** — e.g., call agent endpoints to augment workflow decisions

Simply connecting N8N to third-party services (Slack, Gmail, etc.) alone is not sufficient — you must bring in something you built.

Quality & Completeness Standards

1. **Meaningful Business Logic**

    - The workflow solves a real problem or automates a genuine task
    - Not a "Hello World" demo — something you'd actually use or show in a portfolio
    - Example: "Summarize daily Slack threads and email them to the team" (good) vs "Send a message to Slack" (too simple)

2. **Error States Handled**

    - Test what happens when a step fails
    - Document the expected failure modes

- Example: "If the API rate limit is exceeded, the workflow pauses and retries in 5 minutes"

3. **Data Validation**

- Validate inputs before they're processed
- Handle edge cases (empty lists, null values, unexpected formats)
- Example: Check that an email address is valid before sending

---

Submission Checklist

Before submitting, verify:

- [ ] Workflow includes at least 3 of the 6 complexity features listed above
- [ ] Workflow integrates with your chatbot backend, database, or AI models
- [ ] Workflow solves a real problem, not a trivial demo
- [ ] Error handling is implemented
- [ ] Workflow has comments explaining key steps
- [ ] You've tested the workflow end-to-end
- [ ] You can explain the purpose and flow in 2–3 minutes

Examples of Sufficient Complexity

Good examples:

1. **Daily Standup Automation**

- Trigger: Scheduled for 9 AM weekdays
- Steps: Fetch last 24h messages from Slack → Summarize with Claude → Post summary to a channel → Store in Sheets for history
- Complexity: Scheduled trigger, external integrations, AI model, data persistence

2. **Smart Task Routing**

- Trigger: New task posted to Slack
- Steps: Extract task details → Check complexity via your NL-to-SQL backend → Route to team/queue based on priority → Log result to database
- Complexity: Multiple integrations, conditional routing, database I/O

3. **Document Processing Pipeline**

    - Trigger: PDF uploaded to Google Drive
    - Steps: Extract text → Send to your RAG backend for processing → If high confidence, store summary in database → Else, flag for manual review via email
    - Complexity: Multiple integrations, conditional branching, error handling, your backend

4. **User Onboarding Workflow**

    - Trigger: New user registered via your chatbot
    - Steps: Fetch user data from Supabase → Send welcome email → Create task in project management tool → Log event to analytics
    - Complexity: Database read, multiple external APIs, data transformation

Insufficient examples (avoid):

- Single Slack message send
- Just forwarding one API response to email
- Linear workflow with no branching or transformation
- No interaction with your own systems